Autoscaling LLM Inference on Kubernetes: CPU-Driven Elasticity of a llama.cpp Service under HPA

Student Name 1
Matricola XXXXXXX
Sapienza University of Rome
name1.XXXXXXX@studenti.uniroma1.it

Student Name 2
Matricola XXXXXXX
Sapienza University of Rome
name2.XXXXXXX@studenti.uniroma1.it

Abstract*—We implement Project Option 4: a self-managed Kubernetes cluster on AWS EC2 (`kubeadm`, no EKS) whose Horizontal Pod Autoscaler (HPA) scales a Dockerized application under different workload conditions. The application under test is a small-parameter LLM (Qwen3.5-0.8B) served by llama.cpp inside a Kubernetes pod behind a thin FastAPI proxy, and we evaluate whether a CPU-based HPA is a sound elasticity mechanism for LLM inference, and at what cost. The experimental deployment runs on six `t3.medium` workers, a bench sized so the HPA can reach its six-pod ceiling, targeting 60% CPU between 1 and 6 pods. Test A drives it with a sawtooth user profile (N=20) and the HPA scales out 1→2→4→6 and back 6→5→4→3→2→1 in every run: a repeated bidirectional elasticity cycle that includes mid-run scale-in, not only a single out-and-back. Raising the HPA ceiling from 4 to 6 pods (exp4/exp6, N=20 per level) keeps availability ≥0.76 and keeps exp6's p95 below 100 s, though saturation persists: at level 50 the six-pod deployment still reports a 16.6% error rate, while processing 7× the requests of the four-pod one (1,299 vs 178) by absorbing the queue instead of dropping it. Attributing end-to-end delay per request shows that the llama decode dominates (≈40 s) while the orchestrator and transport path contribute only ≈11 ms, so the bottleneck is the model container, not Kubernetes. Size-isolated runs (Test C, six fixed pods) isolate the effect of request size and expose cross-size interference: a small request takes 7.7 s in isolation but 10.9 s when mixed with medium and large ones (a 1.4× penalty), with zero errors across 5,227 requests. A bursty workload (Test D) shows the HPA reacts to a sudden 6× step in users within 27 s (CPU, reported as a fraction of the pod's 1800 m request, jumps 0%→86–106%) with zero errors over 145 requests, although short bursts are absorbed by the service queue rather than rejected. Projected over six months, the service at its operating footprint (1 master + 2 workers, HPA 1–2 pods) costs $513, versus $3,787 for an equivalent AWS Lambda deployment (≈7×); the six-worker bench used for these experiments costs $1,327, still below the no-elasticity single instance that matches it.

I. Introduction

LLM inference is less commonly evaluated under reactive autoscaling. Each request is long-lived, compute-bound, and variable in cost: on CPU-only hardware a single generation occupies a core for tens of seconds while the model decodes one token at a time. Whether a reactive autoscaler can serve such a workload usefully is not obvious. HPA-like controllers react to a measured signal (here CPU), but the signal itself is the work being done, and the work takes longer than the reaction horizon. This report does not ask whether Kubernetes can add pods; it asks whether the CPU of an LLM service is a useful elasticity signal, what the deployment is capable of at its limits, and what it costs. The assignment is Project Option 4: build a Kubernetes cluster on plain EC2 (no EKS), enable pod autoscaling, and show that a Dockerized application scales out and in under different workload conditions. Our Dockerized application is the LLM inference service; Option 4's web application is our `/generate` API.

We report a single campaign with one configuration, and the shape of the load is the spine of this paper. Test A (Sec. VI-A) drives the deployment with a sawtooth user profile and shows that the HPA deterministically scales out 1→2→4→6 and back 6→5→4→3→2→1 in every one of N=20 runs. Repeated mid-run scale-in is less commonly reported; many published profiles do not include a long idle period, and fewer still a descent that brings the cluster back down while the run is still live. Delay attribution over per-request timestamps establishes where the time goes and isolates the bottleneck. Test B (Sec. VII) varies the HPA ceiling between 4 and 6 pods and shows monotonic improvement in latency, errors and availability as the ceiling rises.

That experiment produced the result we expected to see confirmed rather than discovered: the elasticity variable is the per-request service time, set almost entirely by the model, with the orchestrator contributing a constant ≈11 ms. Varying request size isolates this cleanly: a small generation takes 7 s, a large one 52 s, at the same pod count, and the mixed workload shows the queueing penalty a long tail imposes on short requests. A burst experiment shows the practical edge of the design: a sudden 6× step in users is absorbed without a single error, because the queue drains during the following lull.

Section II gives the background, Sec. III positions the work, Sec. IV describes the application, Sec. V the methodology, Sec. VI the sawtooth elasticity experiment, Sec. VII Test B at N=20 and the remaining tests, and Sec. VIII the cost evaluation.

II. Background

A. Kubernetes and the Horizontal Pod Autoscaler

Kubernetes schedules application instances (pods) on worker nodes. The Horizontal Pod Autoscaler (HPA) reads a resource signal, here average CPU utilisation across the pods of a deployment, and, when the utilisation crosses a configured target, changes the number of replicas between a configured minimum and maximum. Scaling is governed by two stabilisation windows: scale-*out* may start within seconds, while scale-*in* is delayed (300 s by default) so that transient dips do not collapse the pool. The signal

comes from the Metrics Server, which reads CPU usage from `cgroup` accounting. Two properties of this mechanism matter for our workload. First, the signal is a *percentage of the CPU request*, not of the physical node: a pod that requests 1800 m (1700 m llama + 100 m proxy) of the 2000 m a `t3.medium` offers will read near 100% whenever the model is decoding, and above 100% when usage exceeds the request. Second, HPA acts on an average and reacts to the recent past, so the latency of the reaction is bounded by how long the CPU has already been saturated.

B. llama.cpp and CPU-based LLM inference

llama.cpp is a CPU-first inference engine for quantised transformer models. Generation is autoregressive: the model emits one token at a time, and each token consumes the full model compute. On two virtual cores, our 0.8B-parameter model (Q6_K quantisation, 791 MB) decodes at ≈26 tok/s (`MEASURE.md`), so a 256-token response takes roughly 10–20 s of sustained single-core work. With `-parallel 2` each instance holds two decode slots, so a pod can serve two generations concurrently, after which requests queue inside the proxy. This is the entire mechanism of our load: CPU is the work, and the work is a long decode.

C. The experimental environment

The deployment runs on AWS Academy Learner Lab, a sandbox where we self-enforce stricter caps (≤8 instances, ≤31 vCPU, instance size ≤medium) below the lab's hard limits to eliminate the risk of account deactivation for quota breach; the caps are enforced by the course scripts (`guards.sh`/`01-launch.sh`) and fail the launch before any resource is created. Sessions last four hours, credentials are temporary and region-scoped (us-east-1 or us-west-2), and the lab auto-restarts instances left stopped, so every experiment is a fresh cluster built from scripts (`infra/`) and torn down afterwards. Load generation must stay inside the AWS perimeter (Learner Lab guidance); all client-side figures below are intra-region values and lower bounds on what an external user would see.

III. State of the Art

We position our results against the published literature on autoscaling LLM inference and on the economics of provisioned versus serverless compute, and state at the end which of our findings confirm prior work and which do not.

A. Autoscaling LLM inference

Most published LLM serving work assumes GPUs and measures token throughput and batch efficiency rather than elasticity under a load profile; a CPU-only deployment with a single small model is closer to the classic capacity-planning setting. The general lesson we take from the serving literature is that service time is a first-class workload parameter: per- request duration varies by an order of magnitude with the requested output length, which is precisely the variable that makes the product "arrival rate × service time" (Little's Law [1]) the correct capacity input rather than arrival rate alone.

B. Economics: provisioned versus serverless

Adzic and Chatley [2] report 66–95% savings moving low-duty-cycle enterprise workloads to serverless, and Eivy and Weinman [3] show that the benefit depends on execution behavior and volumes and can reverse as traffic grows. Our R4 evaluation (Sec. VIII) finds the same trade-off on an LLM workload: a serverless deployment is roughly 7× the cost of the autoscaled service footprint ($3,787 vs $513) because long CPU-bound inferences are billed at serverless unit prices for every second of compute; Lambda is also ≈2.9× the six-worker bench ($1,327), and a peak-sized single instance without elasticity costs more than either EC2 configuration.

C. Position of this study

Three of our results restate known ones on a new workload: reactive autoscalers need headroom and react on a delayed signal; serverless loses to provisioned for long steady compute; and the bottleneck of a distributed service is usually not the orchestrator. Two are less standard. Repeated scale-in while a run is still live (the cluster stepping back down 6→5→4→3→2→1 mid-sawtooth, not just after a final idle drain) is less commonly reported; Test A demonstrates it in every replicate. And the ≈11 ms orchestrator share of a 40 s request is a direct measurement, using a timing header added to the proxy, of the claim that Kubernetes itself is cheap next to the work it schedules.

IV. Application

A. Architecture

A client sends an OpenAI-compatible `POST /generate` request to the FastAPI proxy inside a pod. The proxy streams the request to a llama-server sidecar on `localhost:8080`, which decodes the model and streams tokens back. A shared `emptyDir` volume carries the quantised GGUF model, placed there by an `initContainer` before the main containers start. Each pod therefore requests 1700 m CPU (llama) + 100 m CPU (proxy), which fits one `t3.medium`; up to six pods spread across six workers. The HPA targets 60% average CPU with `minReplicas` 1 and `maxReplicas` 6. A NodePort service exposes port 30080. The load generator runs on a separate `t3.micro` inside the AWS perimeter.

B. Workload

Three request classes span the resource-usage space by capping the generated output length: `small` (`max_tokens` 32), `medium` (128) and `large` (256). Output length, not input size, is the variable we vary, because on this workload it is the quantity that multiplies decode time. The mix workload draws a size per request with probabilities 0.5/0.3/0.2 (small/medium/large).

TABLE I
MAPPING OF THE EVALUATION ONTO THE STANDARD PROCEDURE FOR MEASUREMENT-BASED CLOUD SERVICE PERFORMANCE EVALUATION

Step	Where addressed
1. Purpose, scope	Sec. V; client and server side
2. Features	LLM inference pod; auth, data store and front end excluded by assumption
3. Metrics	Sec. V-A
4. Tool	Locust 2.46 headless with custom `LoadTestShape` profiles (sawtooth, burst)
5. Design	Sec. V-C; sawtooth, intensity sweep, size isolation, burst; N=20 (Test A and Test B), N=10 (Test C), N=3 (Test D)
6. Environment	Sec. V-E
7. Execution	Sec. VI, Sec. VII

C. Instrumentation

Two measurements are measurement infrastructure rather than application logic. First, the proxy adds an `X-Upstream-Ms` header carrying the proxy→llama round-trip, so the per-request delay can be split as

$$\text{orchestrator+transport} = \text{total} - \text{upstream},$$

where *total* is the client-side response time and *upstream* is the header. Second, a collector on the master (`infra/collect.sh`) samples `kubectl top pods`, replicas, HPA state and events every minute (20 s in the burst test) into per-run CSV files, aligned by UTC timestamp with Locust's per-request detail log.

V. Performance Evaluation Methodology

Table I maps the evaluation onto the standard procedure for measurement-based performance evaluation of cloud services.

A. Metrics

User-oriented metrics come from the generator: response time (median, p95), successful requests, achieved throughput, and error rate. *System-oriented* metrics come from the collector: pod count, HPA state, CPU utilisation and autoscaling events. Availability is reported in two forms: end-to-end success rate (2xx over all attempts) and, where relevant, reachability of the service. Throughput is successful requests per second.

B. Tool selection

We chose Locust 2.46: Python (already the project language), headless with CSV output (`locust_stats.csv`, `locust_failures.csv`), a `LoadTestShape` API for custom profiles, and per-request hooks that write the timing detail used in the delay split. The generators are *closed-loop*: a fixed number of simulated users issues requests back-to- back, so achieved rate is a consequence of concurrency and service time rather than an input. We accept this because the quantity we control is the number of concurrent users, which is how a real audience behaves.

C. Load profiles

Four profiles cover the questions. **Test A** (elasticity) drives a sawtooth: warm-up, ramp to 20 users and hold (scale-out to 4 pods), ramp to 50 and hold (scale-out to the 6-pod ceiling), then descend through 35 and 10 users (scale-in steps) before an idle drain; each descent leg is held long enough for the HPA's 300 s scale-down stabilisation to fire, so scale-in is observed mid-run; `ramp_shape.py`. **Test B** (capacity) holds a fixed user count (10/20/30/40/50) with a 2-minute steady phase, re-run at N=20 runs per level, superseding the original N=5 Test B, under HPA ceilings of 4 (exp4) and 6 (exp6); `exp-b.sh`. **Test C** (size isolation) fixes the workload size per run (small/medium/large/mix) at 20 users with a 3-minute steady phase, 10 runs per size, on six fixed pods (12 parallel decode slots); `day-run.sh`. **Test D** (burst) alternates a 2-user baseline with 12-user bursts, 120 s normal / 60 s burst, two cycles per run, N=3; `burst_shape.py`.

D. Service level objectives

We fix the following objectives before the results, at the level a small internal text-derivative service could commit to:

- **Availability** $\geq$ 99% end-to-end success at design load ($\leq$20 concurrent users).
- **Error rate** below 10% at design load.
- **Interactive latency:** p95 below the 300 s proxy timeout for the small class, the class a user waits on synchronously.
- **Elasticity:** scale-out to the ceiling within 60 s of the CPU target being crossed, and scale-in back to one pod after the load recedes.

Section VII-D reports compliance. We state these before the results because an objective chosen after seeing the data is not an objective.

E. Experimental environment

All runs used 1 master `t3.small` and 6 workers `t3.medium` (Amazon Linux 2023, Kubernetes v1.36, Flannel, Metrics Server). Test A and the Test B's ceilings (exp4, exp6) ran on this same six-worker cluster, differing only in the HPA `maxReplicas` ceiling (6 for Test A and exp6, 4 for exp4). The model is `unsloth/Qwen3.5-0.8B-MTP-GGUF:UD-Q6_K_XL`, served by `ghcr.io/ggml-org/llama.cpp:server` build 10380 with `-threads 2 -parallel 2 -reasoning off`. The load generator ran on a `t3.micro` in the same region. Every experiment ended with a full teardown (`just cluster-down`).

A robustness note: llama-server (build 10380) degrades into a deadlock under sustained concurrent load (node

CPU falls to single digits while every request times out at the proxy), so each Test A run starts from a freshly restarted pod (`FRESH_POD`); a single generation returns in 5 s after the restart. The restart doubles as a controlled cold start: every run begins with a cold model and builds up from an empty cluster.

All conclusions apply to a 0.8B CPU-bound model on t-series instances; GPU serving, batched inference or a larger model would change service time, hence the elasticity axis, though not the method. Our self-enforced caps (8 instances / 31 vCPU) bound Test B to N=20 and six workers, and the lab's temporary, region-scoped credentials meant runs span two regions (us-east-1, us-west-2).

VI. Elasticity Under the Sawtooth

A. Elasticity under a sawtooth (Test A)

Test A drives the deployment with a sawtooth user profile (warm-up, hold at 20 users, hold at 50 users, descend through 35 and 10 users, idle drain), with the HPA ceiling at 6 pods. Twenty replicates (N=20), all complete (`interrupted`=0). In every run the HPA scaled out 1→2→4→6 as CPU crossed the 60% target under the growing load, and then stepped back down 6→5→4→3→2→1 as each valley and the final drain brought CPU below target long enough for the HPA's 300 s scale-down stabilisation window to expire. Kubernetes recorded the corresponding `SuccessfulRescale` events in every run. Across the campaign: 12,144 requests, 116 errors (1.0%), all 504-generation timeouts at the 50-user peak; the per-run error rate stayed below 0.5%.

Fig. 1 plots replicas and CPU against time (the N=20 average). The repeated out-and-back cycle is the elasticity evidence: scale-out is observed within the first 60 s sampling window in every run (below the collector's 60 s resolution, so the true delay is bounded above by 60 s rather than measured directly. The full scale-in back to one pod takes a mean of ≈740 s (range 352–1,614 s) from the moment CPU falls below the 60% target: roughly the first 300 s is the HPA's scale-down stabilisation window, after which the deployment steps down through 6→5→4→3→2→1. Because scale-in fires mid-run, the descent is observed in every replicate rather than only at the end of a run.

Every run starts from a *cold* pod: as the robustness note above describes, we restart llama-server before each replicate, so the model is loaded from disk at run start and the first seconds of the warm-up absorb the cold-start transient. The measurements therefore capture full cold-to-warm autoscaling behaviour, and the steady-state plateaus we report are reached from a genuinely empty cluster each time.

B. Where the response time goes

The proxy's `X-Upstream-Ms` header splits client wait time into llama decode (upstream) and orchestrator+transport. Across the Test B runs the split is stable:

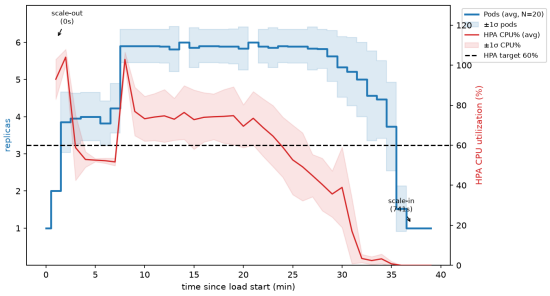

Fig. 1. Test A: pod replicas and CPU% against time (N=20 average with ±1σ band). The sawtooth scales out 1→2→4→6 under the ramp and steps back down 6→5→4→3→2→1 during the valleys.

TABLE II
Test B: error rate, availability and p95 by ceiling and level

Ceiling	Level	Errors %	Availability	p95 (s)
exp4 (max 4)	10–50	4–27	0.73–0.96	51–101
exp6 (max 6)	10–50	0–24	0.76–1.0	30–98

from exp4 (HPA max 4) and exp6 (HPA max 6) the total and upstream means coincide at 44.6 s / 44.6 s (exp4) and 40.1 s / 40.1 s (exp6), leaving an orchestrator+transport share of 10.9 ms and 10.6 ms respectively. The hot spot is the model container; Kubernetes scheduling, networking and the proxy contribute ≈11 ms, three orders of magnitude below the work.

C. Compliance with the elasticity objective

Test A meets the elasticity SLO outright: scale-out to the six-pod ceiling within the 60 s horizon in every run, and scale-in back to one pod after each valley (Sec. VI-A). The capacity SLOs are evaluated on Test B: the 10% error-rate target is met by exp6 only at levels 10 and 30 (2.8% and 0%) and missed elsewhere: 24.2% at level 20 (a cold-start artifact discussed in Sec. VII-D), 13.2% at level 40, and 16.6% and 26.4% at level 50 for exp6 and exp4. We return to this in Sec. VII-D.

VII. Test B at N=20: the HPA ceiling (exp4 and exp6)

A. Capacity at ceilings 4 and 6

Test A established that the HPA scales to the six-pod ceiling under a sawtooth. Test B asks whether the ceiling itself matters: the original capacity sweep (fixed intensities 10–50 users, N=5 per level) was re-run at N=20 per level under `maxReplicas` 4 (exp4) and 6 (exp6). Both runs used the same six-worker cluster, so the only difference is the ceiling, and the same mechanism that produced the sawtooth of Test A now responds to sustained load (Fig. 2).

Table II summarises. The six-pod ceiling dominates the four-pod one at every level; exp6 reaches zero errors at level 30 and an availability of 1.0. The most striking row

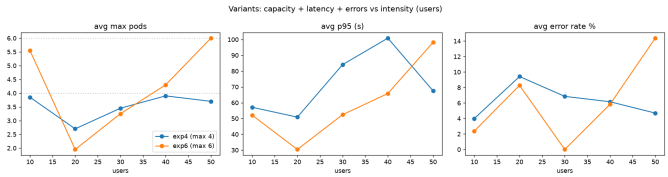

Fig. 2. Test B: p95 latency and error rate against level for exp4/exp6. More pods, lower tail and fewer errors.

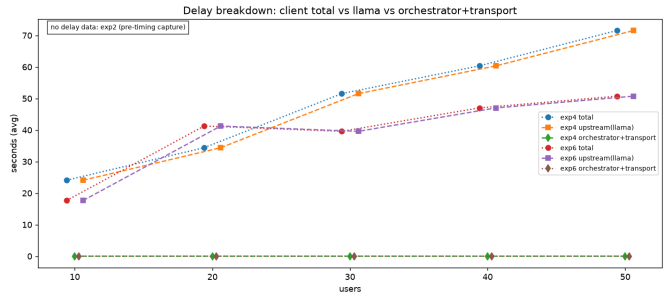

Fig. 3. Delay split (total / upstream / orchestrator) per level. The orchestrator share is ≈11 ms; llama decode dominates. A small x-position jitter (±0.6 users) is applied so the overlapping total and upstream lines remain separately visible.

is level 50: exp4 completes 178 requests at 26.4% errors, exp6 completes 1,299, seven times, at 16.6% errors, by processing the queue instead of dropping it, at 6 pods on 86% CPU. Fig. 2 shows the capacity curves, and Fig. 3 the delay split.

A methodological caveat applies to the Test B runs: there is no warm-up phase. Runs ramp users immediately (`-r 5`), so the figures include cold starts and the HPA cold-starting pods mid-run after scaling down between runs (level 30: 6 of 20 runs started at one pod). Delay and pod figures are therefore full autoscaling behaviour, not clean steady state; Test A, which has an explicit warm-up and idle drain, is the clean case.

B. Effect of request size (Test C)

Test C fixes the workload size per run at 20 users (3-min steady, 10 runs per size, on six fixed pods, 12 parallel decode slots). Table III reports the outcome.

All 5,227 requests complete with **zero errors** and no empty run: the twelve parallel slots (6 pods × `-parallel 2`) remove the queue collapse that a two-slot deployment showed at the same intensity. The service time scales with the request: throughput drops from 1.56 req/s (small) to 0.25 req/s (large) while p50 grows from 7.4 s to 52.4 s, and the orchestrator adds only 9–10 ms. **The bottleneck is therefore the number of parallel decode slots, not raw CPU capacity:** raising the slot count from two to twelve turned a collapsing queue into zero-error service at the same 20-user intensity, while the per-request service time remains dominated by llama's sequential token decode.

TABLE III
TEST C: SIZE-ISOLATED WORKLOAD @20 USERS, SIX FIXED PODS (N=10 PER SIZE)

Size	Req	Req/s	p50 (s)	p95 (s)	Err %	Orch. (ms)
small	2793	1.56	7.4	15.7	0.0	8.9
medium	910	0.51	30.1	59.4	0.0	9.4
large	441	0.25	52.4	120.7	0.0	10.2
mix	1083	0.61	25.2	65.2	0.0	9.5

TABLE IV
CROSS-SIZE INTERFERENCE: P50 LATENCY ISOLATED VS INSIDE THE MIXED WORKLOAD

Size	Isolated (s)	Mix (s)	Penalty
small	7.7	10.9	1.4 ×
medium	30.1	27.4	1.0 ×
large	47.6	48.0	1.0 ×

The mix still exposes cross-size interference: Table IV shows a *small* request 1.4 × slower inside the mix (p50 10.9 s vs 7.7 s isolated) because it queues behind longer ones; medium and large are unaffected (1.0 ×). The penalty is much weaker than the 3.5 × measured with two slots, because the larger slot pool drains the mixed queue faster. Fig. 4 shows latency, delay breakdown and scaling per size.

One operational caveat: `llama-server` leaks memory under sustained load (2.9 GiB in 45 min, limit 3 GiB) and stops completing requests once it degrades. The campaign restarts the pods every five runs (and drains 90 s between runs) to stay out of that region; a production deployment would need a leak fix or a memory-triggered restart policy.

C. Response to bursty load (Test D)

Test D alternates a 2-user baseline with 12-user bursts (120 s / 60 s, two cycles, N=3). Table V reports per-run figures.

All 145 requests succeeded. The HPA reacted to the first burst in 27 s, reading CPU 0%→86%→106%, utilisation reported as a fraction of the pod's 1800 m CPU request (1700 m llama + 100 m proxy, Sec. II), so values above 100% are expected whenever a pod decodes at more than its request, and issuing `SuccessfulRescale` to two pods (Fig. 5); CPU falls to 49–53% between bursts but the HPA holds two pods, because its 300 s scale-*in* stabilisation window exceeds the 120 s lull, so scale-*in* is not observable within a run. The informative contrast: 50 users *sustained* under the six-pod ceiling produce 16.6% errors (exp6, level 50) as the queue pushes a tail past the 300 s proxy timeout, while a 60 s burst at 12 users produced none, because the following lull drains the queue before the timeout; short bursts are absorbed by the service queue, sustained load is not. Test C isolates a light class (small, 32 tokens) at 20 users and shows zero errors because each request is short, on the six fixed pods used there. Sustained mixed load,

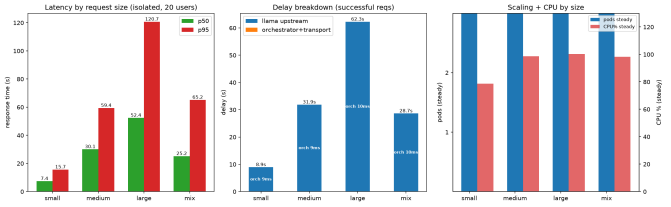

Fig. 4. Test C: latency and delay breakdown per size class. Small is served fast in isolation but is penalised 1.4 × inside the mixed queue; all classes complete with zero errors on six fixed pods.

TABLE V
TEST D: BURSTY WORKLOAD (N=3)

Run	Req	Err	Req/s	p50 (s)	p95 (s)
1	42	0	0.117	24.0	56.0
2	48	0	0.135	21.0	58.0
3	55	0	0.157	20.0	52.0
Total	145	0	–	–	–

bursty load with idle recovery, and light isolated load are different points on the same queueing curve, not conflicting measurements. Pod memory grew from 0.4 to 2.3 GiB across the three runs; the leak documented in Sec. IX stayed below the degradation threshold, so the bursts still completed with zero errors.

D. Availability and SLO summary

Table VI evaluates the objectives of Sec. V-D.

The verdict is mixed in an informative way. Elasticity, the behaviour the architecture exists to provide, passes cleanly in both directions, and availability at design load passes (100% on the warm six-pod Test C configuration at 20 users). The error-rate objective fails even at design load in the elastic variant: exp6 at level 20 reports 24% errors, a cold-start artifact of the no-warm-up protocol (that level completed only 99 requests across 20 runs, ≈5 per run) rather than a steady-state capacity limit, since the warm six-pod configuration at the same intensity is error-free. Interactive latency passes for the small class (15.7 s). The high-load p95 row passes numerically against the 300 s bound (66–101 s)*, but with an asterisk: at the ceiling the tail is cut short by timeouts, and the error-rate SLO, 26.4% (exp4) and 16.6% (exp6) at 50 users, is the binding constraint, not latency. The deployment is correct and elastic; its ceiling is the number of workers.

VIII. Cost Evaluation

Per recommendation R4 we project six months of operation and compare against alternatives achieving the same objective. Every input is measured rather than assumed (instance prices, 24/7 runtime; ‘r4_cost.py‘).

A. Method

The cluster of Sec. V-E is a bench: six workers exist to give the HPA headroom for the 1→6 elasticity cycle and

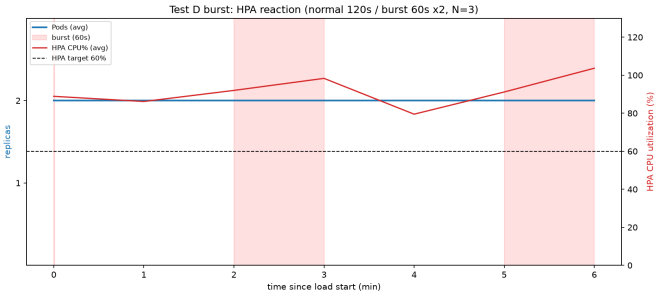

Fig. 5. Test D: HPA reaction to bursts. CPU exceeds the 60% target during each burst and the HPA scales to two pods.

TABLE VI
COMPLIANCE AGAINST THE OBJECTIVES STATED IN SEC. V-D

Objective	Target	Measured	Verdict
Elasticity scale-out	≤60 s	within first 60 s sample (Test A)	pass
Elasticity scale-in	after load recedes	every run, ≈740 s mean (Test A)	pass
Availability (2xx) @20u	≥99%	100% (Test C, six pods)	pass
Error rate @20u	<10%	24% (exp6 level 20)	**fail**
p95, small class	<300 s	15.7 s (Test C)	pass
p95 @40–50 users	<300 s	66–101 s (exp4/exp6)	pass*

Test B's ceilings, and the lab caps bound the campaign. R4 prices the service, not only the bench: we cost two EC2 footprints for six months, plus a no-elasticity single instance for each and a serverless alternative. The service footprint is what the deployment would run in production: 1 master `t3.small` ($0.02/h) + 2 workers `t3.medium` ($0.042/h each), a 40 GB gp3 root volume each, with the HPA placing between 1 and 2 pods on those two workers (4 vCPU of peak compute). The bench is the six-worker cluster actually used for every measurement (12 vCPU of peak). The single instances match each footprint's peak (a `t3.xlarge` at 4 vCPU, an `m5.4xlarge` at 16 vCPU), and the Lambda function (2 GB memory, 40 s mean invocation, 2.84M invocations per six months) is costed once.

B. Results

The service footprint is cheapest ($513.43; master $106.86 + two workers $406.57; the test-only load generator adds $67.41 and is excluded from the comparison). The single `t3.xlarge` costs more ($748.53) for the same 4 vCPU of always-on capacity: that capacity is bought as two `t3.medium` plus a small master, at a lower on-demand per-vCPU price than one `t3.xlarge`. Both EC2 options are billed 24/7 in this projection, so the gap is an instance-purchasing effect rather than an elasticity saving; the elasticity this project demonstrates is operational (pods scale 1→2 on the same instances), not a reduction in

TABLE VII
SIX-MONTH COST PROJECTION (SERVICE FOOTPRINT AND MEASUREMENT BENCH)

Solution	Compute	Total 6 mo
Service footprint (1 master + 2 workers, HPA 1–2)	455.83	**$513.43**
Single t3.xlarge (4 vCPU, no autoscaling)	729.33	$748.53
Experiment bench (1 master + 6 workers, HPA 1–6)	1,192.18	**$1,326.58**
Single m5.4xlarge (16 vCPU, no autoscaling)	4,207.68	$4,226.88
AWS Lambda (same AI app)	3,786.67	$3,787.24

running instances. The footprint is the minimum always-on configuration for the cost comparison; a deployment that must hold the design-load error SLO at $\leq$20 users needs more than the two pods it can host, so the bench below is the SLO-compliant scale. Lambda is 7× the footprint ($3,787.24): a long CPU-bound inference is billed at serverless unit prices for every second of decode, so a workload whose invocations last tens of seconds defeats the serverless pricing model.

The bench tells the same story at the other scale. Even the six-worker cluster actually used here ($1,326.58) undercuts its no-elasticity single-instance equivalent, an `m5.4xlarge` at $4,226.88, by ≈3×; Lambda remains ≈2.9× the bench. Two footprints, two no-elasticity singles, and one serverless option therefore order the same way: provisioned elasticity is cheapest, peak-sized single-node capacity is the most expensive, and per-second-billed serverless loses to both because each invocation is a long CPU-bound decode.

The comparison is best read as a capacity decision, not a duty cycle. At the design-load peak (two pods, 4 vCPU) the autoscaled footprint is cheaper than the peak-sized instance on on-demand prices, and the `t3.xlarge` has no way to express the pod-level elasticity the deployment shows under load (Sec. VI-A); scaling instances down when idle would extend the same logic further, but that needs a node autoscaler, which we do not implement. The serverless alternative is the mirror image: at low and bursty volumes Lambda is competitive, but its unit cost per second of compute is so high that sustaining 0.2 req/s of 40 s LLM decode for six months ($\approx 1.1\times10^8$ s of compute) reaches $3,787. The crossing point is far below the utilisation a production service would target, so for this workload, long, CPU-bound, and always-on, provisioned capacity wins. All prices are on-demand us-east-1 list prices and exclude data transfer, which is identical across solutions.

IX. Limitations and Threats to Validity

No warm-up in Test B. Runs ramp users immediately (`-r 5`), so the data include cold starts and the HPA cold-starting pods mid-run after scaling down between runs (level 30: 6/20 runs started at one pod). Delay and pod figures are therefore full autoscaling behaviour, not clean steady state; Test A, which has an explicit warm-up and idle drain, is the clean case. This is stated explicitly rather than hidden.

Test C large was underpowered at two slots. The earlier two-slot run collected ten requests in ten runs (seven empty) because the protocol is shorter than the 90–120 s service time. The six-pod run reported in Sec. VII-B removes this: 441 large requests across ten full runs. Test C is therefore a *capacity* measurement at a fixed configuration (six pods, HPA pinned); reactive autoscaling behaviour is covered by the exp4/exp6 variants, Test A and Test D.

llama.cpp memory leak under sustained load. `llama-server` grows to 2.9 GiB (of a 3 GiB limit) after 45 min of continuous load and stops completing requests once degraded; no reset command exists in the server API, so a fresh process is required. The campaign stays out of the degraded region with a 90 s drain between runs and a pod restart every five runs. Without a leak fix or a memory-triggered restart policy, sustained campaigns will eventually stall.

No scale-in observable in Test D. The HPA's 300 s scale-in stabilisation window exceeds the 120 s lull, so the burst experiment cannot show a step down in replicas. The evidence for scale-in rests on Test A.

Error rates are real saturation data. At level 50, exp4 (four-pod ceiling) shows 26.4% errors and exp6 (six-pod ceiling) still shows 16.6%; these are reported as the consequence of the pod ceiling relative to offered load (503 busy, 504 timeout, 502 upstream), not hidden or "fixed".

Single lab, single model, CPU only. All conclusions apply to a 0.8B CPU-bound model on t-series instances. GPU serving, batched inference or a larger model would change service time, hence the elasticity axis, though not the method.

Learner Lab constraints. Hard caps (8 instances / 31 vCPU) bound the Test B to N=20 and six workers; a full factorial would need more headroom. Credentials are temporary and region-scoped; runs span two regions (us-east-1, us-west-2).

X. Conclusion

We evaluated a CPU-based HPA as the elasticity mechanism for an LLM inference service on AWS Learner Lab, with a six-pod ceiling. Test A drives the deployment with a sawtooth profile and the HPA scales out 1→2→4→6 and back 6→5→4→3→2→1 in every one of N=20 runs, including mid-run scale-in with a mean latency of ≈740 s, the HPA stabilisation delay. Attributing client wait time per request showed that llama decode accounts for the whole latency budget while the orchestrator and transport contribute ≈11 ms, so the bottleneck is the model container, not Kubernetes.

The Test B ceiling experiment asked whether the ceiling, not the autoscaler, limits the service. Raising the HPA ceiling from four to six pods kept p95 below 100 s and availability ≥0.76 at every level, although the ceiling does not remove saturation: at level 50 the six-pod deployment

still drops 16.6% of requests while processing seven times the requests of the four-pod one by absorbing the queue. Size isolation at six fixed pods showed a request's service time scales with its length and that a mixed queue penalises short requests $\approx$1.4 $\times$, with zero errors across 5,227 requests. The burst experiment showed the edge of the design: a sudden 6 $\times$ step in users is absorbed with zero errors across 145 requests because the queue drains during the lull, while the same intensity sustained under the six-pod ceiling (exp6, level 50) reaches 16.6% errors.

Economically, the service at its operating footprint (1 master + 2 workers) costs \$513 over six months against \$3,787 for the equivalent Lambda deployment ($\approx$7$\times$); even the six-worker bench used here (\$1,327) undercuts both its no-elasticity single-instance equivalent (\$4,227 for an `m5.4xlarge`) and Lambda ($\approx$2.9$\times$). For this CPU-bound workload, CPU utilisation proved to be a useful scaling signal, provided the HPA ceiling is set to the worker count the load actually needs. Measured against the objectives fixed in advance, the deployment is elastic and correct; its limits are the number of workers and the patience of the 300 s timeout, not the autoscaler that decides how many pods exist.

References

[1] J. D. C. Little, "A proof for the queuing formula $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.

[2] G. Adzic and R. Chatley, "Serverless computing: economic and architectural impact," in *Proc. 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*, Paderborn, 2017, pp. 884–889.

[3] A. Eivy and J. Weinman, "Be wary of the economics of 'serverless' cloud computing," *IEEE Cloud Computing*, vol. 4, no. 2, pp. 6–12, 2017.

[4] G. Gerganov et al., *llama.cpp* (open-source CPU LLM inference engine), https://github.com/ggml-org/llama.cpp.

[5] The Kubernetes Authors, *Kubernetes Documentation: Horizontal Pod Autoscaling*, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.